

OpenSpec 工作研究

2026-08-19 16:28

Table of Contents

OpenSpec 强在"约束 AI"，而不是"替代工程师判断"

1. 把 AI 放进一个工程框架，让 AI 在每次实施一个新任务时都遵守下面这个工作流：
2. 使用运行时 CLI 动态维护整个工作流状态，并重构用户给 AI 的 prompt。

OpenSpec 工作流简介

OpenSpec 在 AOE3D 上的工作实践

初始化 OpenSpec command - openspec init

初始化主 specs

使用 OPSX skill 执行 OpenSpec 工作流

第一步 执行 opsx:propose

第二步 执行 opsx:apply

第三步 执行 opsx:archive

OpenSpec 工作流优化

先说结论：OpenSpec 的目标是让工程师和 AI 共同维护一个 specification（规格），通过规范化 prompt 来约束 AI 的产出，达到工程化可控、稳定的输出。

OpenSpec 强在"约束 AI"，而不是"替代工程师判断"

为了实现"约束 AI"，它主要做了两件事：

1. 把 AI 放进一个工程框架，让 AI 在每次实施一个新任务时都遵守下面这个工作流：

1. 先提案：propose skill

- 写提案文档 `proposal.md`：说明"为什么要改、改什么、影响什么"。它是变更的入口文档，面向评审和决策，用来交代动机、范围、新增或修改的能力，以及受影响区域。
- 写规格文档 `spec.md`：说明"系统最终必须表现成什么样"。它是行为契约，通常用 Requirement + Scenario 写清楚可验证的需求，不关注代码怎么实现。
- 写设计文档 `design.md`：说明"准备怎么实现，以及为什么这样设计"。它补充技术方案、关键决策、取舍、风险、非目标，适合复杂变更；简单变更可以很短，甚至可能不需要太多内容。
- 写任务拆分文档 `tasks.md`：说明"具体怎么落地"。它把 `proposal/spec/design` 拆成可执行的实现清单，用 checkbox 跟踪进度，偏工程执行，不应替代需求或设计说明。

2. 再实现：apply skill

3. 最后归档：archive skill

2. 使用运行时 CLI 动态维护整个工作流状态，并重构用户给 AI 的 prompt。

纯文本约定只能提醒 AI"应该怎么做"，而运行时状态可以明确告诉 AI"当前能做什么、下一步该做什么"。这使 OpenSpec 不再依赖聊天记忆，而是每次都从本地文件和 schema 重新计算最新状态。

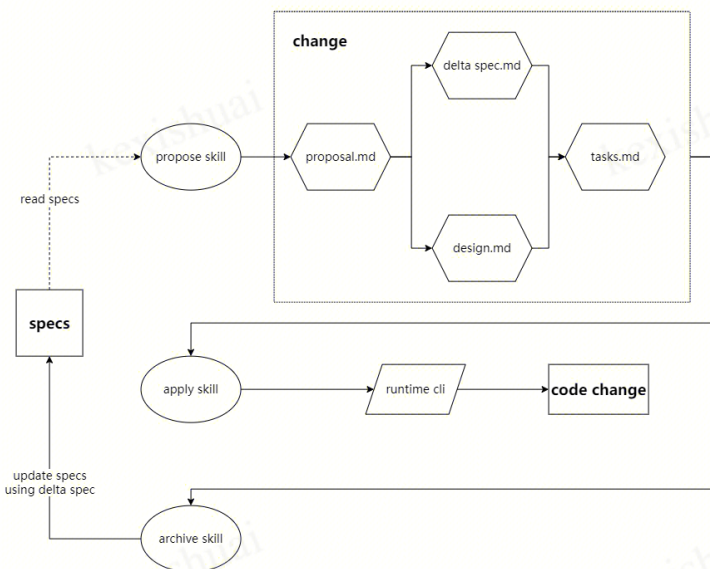
这样带来几个好处：

- 降低幻觉和误操作：AI 不需要凭上下文猜任务进度，而是通过 CLI 获取当前真实状态。
- 支持跨会话协作：即使换窗口、隔天继续，或换另一个 AI 工具，也能从本地状态恢复工作流。
- 减少上下文负担：每一步只注入当前 artifact 所需的模板、规则和依赖信息，避免一次性塞入过多 prompt，让 AI 输入更聚焦、更准确。

以上可能会比较抽象，我后面会详细解释。

OpenSpec 工作流简介

下面是 OpenSpec 的一个工作流展示



但这个工作流会涉及一些关键概念，我们后面再结合 AOE3D 项目实践去讲。

OpenSpec 在 AOE3D 上的工作实践

让我们通过一次使用 OpenSpec 的工作实践，来进一步探索其工作原理。

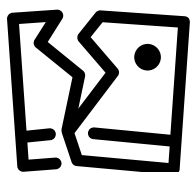
背景：我需要在已有项目 AOE3D 上构建 OpenSpec 环境，并使用 OpenSpec 工作流，为项目实现一个新需求：修改英雄 detail 界面的 UI 展示逻辑，当英雄数据源来自"丝绸之路布阵"，界面需要显示"丝绸之路英雄"标记，并且根据英雄属性显示"援军"标志。

初始化 OpenSpec command - openspec init

openspec init 是一个 command 指令，负责把普通项目变成 OpenSpec 可管理的项目，并把 OpenSpec 的常用流程注册到 AI 编程工具里，搭建脚手架。

主要会在项目目录下生成下面这两块内容：

1. 在 AI 工具例如 .cursor 下生成一系列 OpenSpec 相关的 commands 和 skills。



1. 在项目目录下生成 openspec 路径，里面包含服从 OpenSpec 工作流的固定结构：changes、specs，以及 config.yaml。changes 和 specs 两个路径，以及 config 文件就是整个 OpenSpec 的核心，但到目前为止这些路

径和文件其实都是空的。



- **spec** 是项目的正式规格文档。

它描述某个模块具备什么能力、在什么场景下应该有什么行为。它是 OpenSpec 维护的核心资产，也是项目的"行为事实来源"。spec 通常位于 openspec 根目录下的主 specs，按照功能模块分类：

openspec/specs/<capability>/spec.md



- **schema** 是 workflow 模型配置。

它定义一次 change 应该产出哪些 artifact、这些 artifact 的依赖顺序、每个 artifact 写到哪里、用哪个 template、有什么 instruction，以及 apply 阶段需要哪些前置条件。

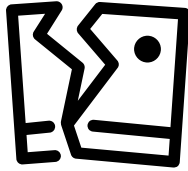
OpenSpec 的 schema 定义的工作流是 proposal → specs → design → tasks → apply。

这个配置最好不要修改，已经是经过大量实践验证的流程。

<https://iwiki.woa.com/p/4025814054>照版

- **artifact** 是在 propose 一个 change 过程中产出的文档。

之前提到的 spec、design、proposal、tasks 文档都属于 artifact。



它由 schema.yaml 文件（在 OpenSpec 的源码中）定义。

下面展示的是 schema.yaml 中对 proposal 的文件定义，比较重要的是 instruction 部分，告诉 AI 需要怎么去生成这个文件：

```
artifacts:
- id: proposal
  generates: proposal.md
  description: 概述本次变更的初始提案文档
  template: proposal.md
  instruction: |
    创建提案文档，阐明为什么(why)需要这次变更。

    章节:
    - **Why(为什么)**: 用 1-2 句话说明问题或机会。它解决了什么问题？为什么是现在？
    - **What Changes(变更内容)**: 以要点列表列出变更。对新增能力、修改或移除要写具体。用 BREAKING 标记破坏性变更。
    - **Capabilities(能力)**: 明确哪些规范(spec)将被创建或修改:
      - New Capabilities(新增能力): 列出要引入的能力。每一项都会成为一个新的 `specs/<name>/spec.md`。使用 kebab-case。
      - Modified Capabilities(修改能力): 列出其“需求(REQUIREMENTS)”发生变化的既有能力。仅当规范层面的行为发生变化时才列。
    - Impact(影响): 受影响的代码、API、依赖或系统。

    重要提示: Capabilities 这一节至关重要。它在提案阶段与规范阶段之间建立了契约。
    在填写之前，请先研究既有规范。
    此处列出的每一项能力，之后都需要一个对应的规范文件。

    保持简洁(1-2 页)。聚焦于“为什么(why)”而非“怎么做(how)”——
    实现细节应放在 design.md 中。

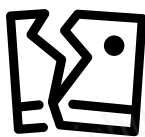
    这是根基——规范、设计、任务都建立在它之上。
  requires: []
```

可以理解为：artifact = OpenSpec 工作流中的一个阶段产物。

- **template** 是 artifact 输出文件的结构骨架。

可以理解为：template = AI 写 artifact 文件时使用的“空白表格/文档骨架”。

它规定文档结构，但不包含项目具体内容。下面是 [proposal.md](#) 的 template：



在初始化时生成的 config.yaml 是空的，只会有一些注释告诉 AI 应该写入相关内容："This is shown to AI when creating artifacts"



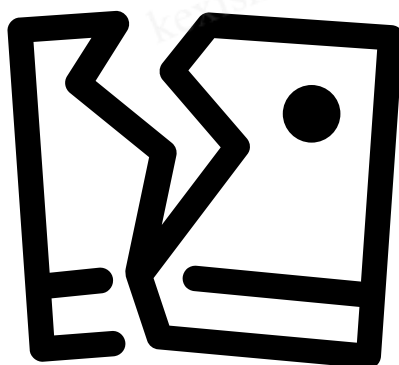
- **config** 是项目级配置。

它是给当前项目提供额外背景和约束，比如：

项目的技术栈、语言、平台要求；各 artifact 要遵守的规则。

可以理解为：config = 当前项目对 OpenSpec 工作流的"本地补充规定"。

这个文件也是工程师能够根据自己项目定制化的主要部分。



注意这里定义的 rules 的名字需要跟 schema 里一一对应，比如上图的 spec 没有与 schema 中的 specs 对应上，因此是无效的。

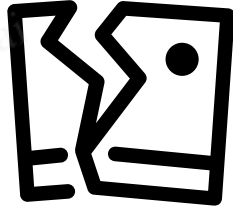
初始化主 specs

openspec init 生成的主 specs 和 config.yaml 都是空的，很明显这还无法让 OpenSpec 正常工作。那么我们此时需要根据项目状态去手动初始化它们。此时就会分两种情况：

- 如果这是个全新的项目，项目路径下甚至是空的（最符合 OpenSpec 工作流的方式）：我们可以让主 specs 保持空的状态，因为后续如果我们一直遵循 OpenSpec 工作流，不断 propose、apply、archive 新的 change，这个主 specs 就会自己生长沉淀起来。我们只需要在 config 文件里写一些项目的基本信息，例如项目定位之类的。
- 如果这是个老项目，已存在大量工程文件（半路才接入 OpenSpec 工作流）：那么在 propose 任何 change 之前，我们需要总结本项目当前的状态，写入到主 specs 里，并在 config 中写明项目信息。（这其实已经违背了

OpenSpec 的工作流，在其工作流里，唯一合法的修改主 specs 是 archive change)

回到我们的项目 **AOE3D**，这是个已有大量代码的大型工程，我们需要在 propose 任何 change 之前，先让 AI 遍历整个项目的状态，写入主 specs。这个处理可以通过调用项目已有的知识库如 OpenViking 实现。



使用 OPSX skill 执行 OpenSpec 工作流

第一步 执行 opsx:propose

在 OpenSpec 工作流中，当我们准备开始一个任务时，第一步通常不是直接让 AI 修改代码，而是先对这个任务执行 propose。该 skill 包含以下步骤：

1. 理解探索目标
2. 读取项目上下文
3. 查询现有主 specs
4. 分析相关代码与文档
5. 总结当前系统行为
6. 给出后续建议

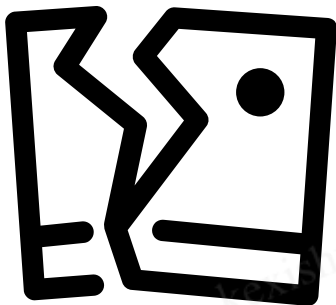
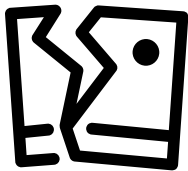
propose 的目标是把需求在实现前结构化下来，让工程师先 review，这样可以在真正改代码之前，确保 AI 和工程师对目标、边界、实现粒度和验收方式达成一致。

回到我们的需求，在进入实现前，尽量准备上下文材料，如策划案、proto、现有英雄数据结构、数据源枚举、UI 展示规则、已有类似逻辑，以及任何已知约束。然后调用 propose skill，把这些材料作为输入交给 AI。

AI 会基于这些信息，按照 OpenSpec 内部的 schema 和 template 产出对应文档：

proposal.md：说明为什么要做这个改动、具体改哪些行为、影响哪些能力或模块。design.md：说明如何实现，例如数据源字段如何判断、UI tag 如何展示、与现有英雄 detail 逻辑如何集成、是否涉及兼容或边界情况。

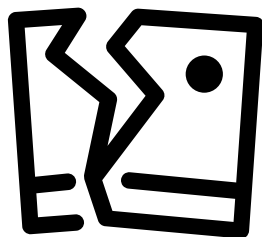
tasks.md：把实现拆成可执行的任务清单，例如梳理数据源字段、修改展示逻辑、补充测试、验证不同来源英雄的展示结果。



这些文档生成后，工程师先进行 review，确认需求理解是否正确、方案是否合理、任务拆分是否可执行。只有 review 通过后，才进入 apply 阶段，让 AI 按任务逐步修改代码。

- **delta spec** 是对 openspec 主 specs 的增量修改，属于 artifact。

delta spec 是临时的，生命周期与所属的 change 绑定，用来描述这次 change。



delta spec 能够描述的操作包含四种：新增需求、修改已有需求、删除已有需求、重命名已有需求。

当这个 change 完成后，对应的 delta spec 会被应用到主 specs 中，这个过程是通过 archive skill 实现的（后面会讲）。

可以理解为：delta spec = "这次 change 对正式 spec 的补丁"。

第二步 执行 opsx:apply

当工程师 review 好了 AI propose 的 change (proposal、design、tasks) , 接下来就是交给 AI 来实现这个任务。

apply skill 主要包含下面的步骤 :

1. 选择目标 change
2. 读取 apply 状态 (用指令 openspec status 调用运行时 CLI)
3. 加载上下文文档
4. 查看待办任务
5. 按 tasks 实现代码
6. 更新任务完成状态
7. 汇总实现进度

"读取 apply 状态"这一步在 OpenSpec 的工作流中非常重要, 也是其对 AI 输出的严格约束的主要体现。

在这里, 我首先要介绍一下 OpenSpec 的运行时 CLI。这个 CLI 会一直跑在后台, 依靠核心层中 Markdown 解析器、Schema 解析器、Spec 校验器等基础设施, 实时计算 OpenSpec 工作流的状态。Agent 通过命令 openspec status --change xxxx 来获取当前 change 进行到哪一步、下一步应该创建什么、哪些文档还被依赖阻塞。

```
f},
"progress": {
  "total": 13,
  "complete": 0,
  "remaining": 13
},
"tasks": [
  {
    "id": "1",
    "description": "1.1 Re-scan exact `UIName.PopupBox_Common`
call sites under `Assets/Scripts/.Lua` and list file:line for
ShowUI/ShowStackUI/Hide/whitelist (verify: list matches design
scope; siblings like CommonRule excluded)",
    "done": false
  }
]
```

然后使用 openspec instruction apply --change xxxx 来组装发给 AI 的 instruction :

```
1 {
2   "changeName": "migrate-popupbox-common-to-msgbox",
3   "schemaName": "spec-driven",
4   "state": "ready",
5   "instruction": "Read context files, work through pending tasks, mark complete as you go.\nPause if you hit blockers or need clarification.",
6   "progressAtStart": { "total": 13, "complete": 0, "remaining": 13 },
7   "contextFiles": {
8     "proposal": ["openspec/changes/migrate-popupbox-common-to-msgbox/proposal.md"],
9     "specs": ["openspec/changes/migrate-popupbox-common-to-msgbox/specs/popupbox-msgbox-migration/spec.md"],
10    "design": ["openspec/changes/migrate-popupbox-common-to-msgbox/design.md"],
11    "tasks": ["openspec/changes/migrate-popupbox-common-to-msgbox/tasks.md"]
12  },
13   "contextSummary": "AOE3D Lua UI: migrate exact PopupBox_Common to PopupBox_MsgBox; bridge ConfirmDialog params; keep nest stack; do not touch
14   "capturedAt": "2026-07-21",
15   "cliCommand": "openspec instructions apply --change \"migrate-popupbox-common-to-msgbox\" --json"
16 }
17
```

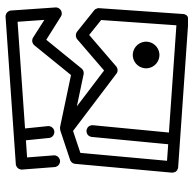
对于不同的命令, 这些 instruction 的格式都不同, 例如 propose skill 组装的 instruction 如下图 :

```

1  {
2    "changeName": "migrate-popupbox-common-to-msgbox",
3    "artifactId": "proposal",
4    "schemaName": "spec-driven",
5    "outputPath": "proposal.md",
6    "description": "Initial proposal document outlining the change",
7    "instruction": "Create the proposal document that establishes WHY this change is needed.\n\nSections:\n- **Why**: 1-2
8    "context": "项目名称: 帝国时代: 远征 (Age of Empires: Expedition / AOE3D) \n项目类型: Unity SLG 手游\n引擎版本: Unity 2021
9    "rules": [
10     "保持提案在500字以内",
11     "始终包含\"非目标\"部分"
12   ],
13   "templateSections": ["Why", "What Changes", "Capabilities", "Impact"],
14   "note": "Captured during propose on 2026-07-21. CLI also printed: Unknown artifact ID in rules: \"spec\"."
15 }
16

```

回到我们的需求实现，在上一步通过 propose skill 获取了这个 change 相关的各种 artifacts，尤其是 tasks 文档后，开始调用 apply skill。在这个过程中，CLI 会读取 tasks 文档，查询进行到第几个 task 了，把跟当前 task 相关的上下文组织起来，加入到给 AI 的上下文中。



这种机制不会一次性把所有任务阶段的要求都塞给 AI，而是按当前 task 注入对应的模板、规则、上下文和已完成依赖，从而把大任务拆成多个受约束的小步骤，降低 AI 跳步、遗漏或幻觉的概率。

第三步 执行 opsx:archive

当 AI 已经成功 apply 完一个 change 后，archive 是非常重要的收尾步骤。因为 apply 只代表实现任务已经完成，并不等于 OpenSpec 的主 specs 已经同步更新，只有 archive 这个 change，才会合并到主 specs。

它相当于把“已经完成的变更”沉淀为正式系统契约，为后续 change 提供准确上下文。

archive skill 包含下面的步骤：

1. 选择已完成 change
2. 校验 change 状态
3. 合并 delta specs
4. 更新主 specs
5. 归档 change 目录
6. 输出归档结果

如果不执行 archive，从 OpenSpec 的视角看，主 specs 仍然停留在变更前状态。这会带来一个问题：后续如果再提出一个与该能力相关的新 change，AI 在 propose 或 specs 阶段通常会参考 openspec/specs 中的正式规格。如果

上一个 change 没有 archive，AI 可能会误以为相关能力还没有实现，进而重复提出已经完成的需求，或者基于过期规格做出错误判断。结果可能是重复工作、规格冲突，甚至后续实现方向发生偏差。

回到我们的需求，当执行 archive skill 后，与我们需求相关的 change 会被转移到 changes/archive 下，而其所属的 delta spec 会被更新到主 specs 中，供其他的 change 读取。



OpenSpec workflow 优化

在对 OpenSpec 的工作原理进行深度剖析后，我们可以发现，OpenSpec 的长效高效运行非常依赖一点：确保主 specs 与项目状态保持同步。

如果主 specs 准确，AI 在 propose、design、apply 等阶段就能基于正确上下文工作；如果主 specs 逐渐过期，AI 就可能基于错误或缺失的信息做判断，导致重复设计、遗漏边界条件，甚至产生与真实代码行为不一致的方案。

但在实际开发中，并不是所有修改都会严格通过 OpenSpec 工作流完成。有些改动可能来自紧急修复、人工直接提交，或者 bug 修复。这些改动如果没有同步回主 specs，就会让 openspec/specs 与项目真实行为逐渐偏离。

因此我提出，在日常使用 OpenSpec 时，可以遵循以下实践：

1. 对于一个相对完整的需求，最好完全按照 OpenSpec 的工作流进行。
2. 在 OpenSpec 工作流中，当一个需求对应的 change 完成后，一定要记得把这个 change archive，沉淀回主 specs。
3. 如果项目有大型版本更新、新模块引入、新技术栈接入、架构调整、目录结构变化等，需要及时更新 openspec/config.yaml 文件。

未来可以进一步优化的一点是：

建立一个专门的同步 skill，定期根据代码提交记录、变更 diff 或 release 内容，识别那些没有通过 OpenSpec 工作流完成的修改，并辅助生成对应的 spec 更新建议。

这个 skill 可以在固定时间段运行，例如每周一次、每个版本发布前，或在重要分支合并后执行。它的目标不是替代 OpenSpec 工作流，而是补齐现实开发中绕过工作流的变更，把这些变化重新同步回主 specs，确保 openspec/specs 始终尽量接近项目真实状态。